DL-RAG-Toolkit Technical Report

DL-RAG-Toolkit — Technical Report

Prepared by: Sameer Ahmed **Role:** AI Engineering Research Intern, Alphatron Technologies (in collaboration with MMSE Lab, NED University) **Repository:** `github.com/SameerAhmedAI/DL-RAG-toolkit`

---

1. Project Overview and Objectives

This project fulfills a two-tier internship task assigned by Alphatron Technologies, requiring demonstrated competency across two levels of AI engineering:

Intermediate Level — Five independent deep learning implementations, each covering a distinct architecture family, trained and evaluated end-to-end with documented results: - Artificial Neural Network (ANN) - Convolutional Neural Network (CNN) - Recurrent Neural Network (RNN) - Long Short-Term Memory network (LSTM) - Transfer Learning (pretrained CNN backbone fine-tuning)

Advanced Level — A Retrieval-Augmented Generation (RAG) chatbot capable of ingesting documents in four formats (PDF, DOCX, TXT, XLSX), answering questions grounded in that content, maintaining multi-turn conversational memory, and refusing to hallucinate when no relevant information exists in the uploaded documents.

The objective was not simply to produce working code, but to produce a defensible engineering artifact: every architectural decision, every dataset choice, every failure, and every fix is documented here so the reasoning can be explained and reproduced by another developer or evaluator.

2. Problem Statement

Two distinct problems are being solved:

Problem 1 (Intermediate Level): Demonstrate practical, hands-on understanding of the core deep learning architecture families that underpin modern AI systems — not just theoretical knowledge, but the ability to implement, train, debug, and evaluate each one, and to articulate *why* certain architectures succeed or fail on certain tasks (most concretely demonstrated by the deliberate RNN vs. LSTM comparison in Section 6).

Problem 2 (Advanced Level): Build a working RAG system that solves the core challenge of grounding a large language model's responses in a specific, private document set — rather than relying on the model's general training knowledge — while correctly handling the two failure modes that plague naive RAG implementations: 1. **Hallucination on irrelevant queries** — answering confidently even when no relevant document content exists. 2. **Context loss on follow-up questions** — failing to resolve pronouns and implicit references in multi-turn

conversations (a real bug encountered and fixed during this project — see Section 9.3).

3. Technology Stack

Intermediate Level

Component	Choice	Reasoning
DL Framework	PyTorch	Consistent OOP model-class pattern across all 5 models, as required by the task
ML Utilities	scikit-learn	Metrics, train/test splitting, preprocessing
Visualization	matplotlib	Training curves, confusion matrices
Datasets	sklearn built-ins, torchvision built-ins	Breast Cancer Wisconsin, FashionMNIST, 20 Newsgroups, CIFAR-10 — all standard, zero-cost, no manual data collection needed
Hardware	CPU only	No GPU available; this constraint directly shaped dataset subset sizes and epoch counts throughout (see Section 9.1)

Advanced Level (RAG Chatbot)

Component	Choice	Reasoning
Orchestration	LangChain	Explicitly required by task scope; handles document loading and text splitting interfaces
Vector Database	ChromaDB	Local, free, persistent — no hosted vector DB cost
Embeddings	sentence-transformers (`all-MiniLM-L6-v2`)	Local, free, CPU-fast, no API cost
LLM	Groq API (`llama-3.3-70b-versatile`)	Zero-cost inference, fast response times, no OpenAI/Anthropic API budget available
UI	Streamlit	Fastest path to a working, screenshot-able demo interface given the project timeline; avoids the cost of building a custom React frontend for a non-production deliverable
Document Loaders	pypdf, docx2txt, openpyxl	Format-specific libraries chosen per file type; XLSX hand-rolled with openpyxl directly rather than relying on a generic LangChain Excel loader (see Section 8.4)

Shared / Infrastructure

Component	Choice	Reasoning
Version Control	Git + GitHub	Standard, required by task deliverables
CI/CD	GitHub Actions	Required by task deliverables — lint + automated test run on push
		Required minimum test coverage on

Testing	pytest	the ingestion layer
Environment	Python venv	Isolated dependency management, avoids global environment conflicts

4. Methodology — Intermediate Level

Every one of the five models follows the same structural pipeline, so results are comparable and the codebase stays consistent:

```
Raw Dataset → Preprocessing → Model Definition (OOP class, PyTorch nn.Module)
→ Training Loop (with validation each epoch) → Evaluation on held-out Test Set
→ Metrics + Plots saved to results_artifacts/ → results.md written with real numbers
```

Two shared modules underpin all five models, avoiding code duplication:

- **`intermediate/common/metrics.py`** — wraps sklearn's `accuracy_score`, `precision_recall_fscore_support`, and `confusion_matrix` into two functions: `evaluate_model()` (runs a full pass over a dataloader and returns a results dict) and `print_evaluation_summary()` (formats that dict for terminal output). Every model's `train.py` calls these two functions identically at the end of training — this is why every results.md has the same metrics structure.
- **`intermediate/common/plotting.py`** — two functions, `plot_training_curves()` (loss + accuracy line charts across epochs) and `plot_confusion_matrix()` (heatmap with per-cell counts). Both save to disk via a `save_path` argument and are called identically from every model's `train.py`.

This shared-module pattern means that understanding *one* model's `train.py` structure means understanding all five — they differ only in: dataset loading, the model class being instantiated, and hyperparameters.

---

5. ANN — Artificial Neural Network

5.1 Dataset

Breast Cancer Wisconsin (Diagnostic), loaded via `sklearn.datasets.load_breast_cancer()`. Chosen because it is a clean, well-labeled, real (not synthetic) tabular medical dataset — 569 samples, 30 numeric features (cell nucleus measurements from digitized images), 2 classes (malignant / benign). Small enough to train fully on CPU in seconds.

Split: 70% train (398 samples) / 15% val (85) / 15% test (86), stratified to preserve the natural class ratio (212 malignant : 357 benign) across all three sets. Features were standardized (zero mean, unit variance) using `StandardScaler`, fit only on the training set to avoid data leakage from val/test statistics.

5.2 Architecture — `intermediate/ann/model.py`

```
Input(30) → Linear(64) → BatchNorm1d → ReLU → Dropout(0.3)
          → Linear(32) → BatchNorm1d → ReLU → Dropout(0.3)
          → Linear(2)
```

Implemented as `ANNModel(nn.Module)`, built dynamically from a `hidden_dims` list (`[64, 32]`) so the architecture can be resized without editing the class body — a small but deliberate flexibility choice for reuse.

Why BatchNorm + Dropout together: with only 398 training samples, overfitting was the primary risk. BatchNorm stabilizes training by normalizing layer inputs; Dropout (0.3) randomly zeroes 30% of activations each forward pass during training, forcing the network to not over-rely on any single feature pathway. Both together — rather than either alone — proved sufficient to prevent overfitting on this dataset (confirmed in results, Section 5.4).

5.3 Training Configuration — `intermediate/ann/train.py`

- Loss: `CrossEntropyLoss`
- Optimizer: `Adam`, learning rate `1e-3`
- Epochs: 30, Batch size: 32
- Hardware: CPU

The training loop (`train_one_epoch()`) and validation loop (`validate()`) are structurally identical across all five intermediate models: iterate batches, forward pass, compute loss, backward pass (training only), accumulate loss/accuracy weighted by batch size, and average over the full dataset at the end of the epoch.

5.4 Results

Metric	Value
Test Accuracy	0.9767
Test Precision (weighted)	0.9776
Test Recall (weighted)	0.9767
Test F1 (weighted)	0.9766

Per-class: Malignant — 100% precision, 94% recall, 97% F1. Benign — 96% precision, 100% recall, 98% F1.

5.5 Observations

Train and validation loss decreased together throughout all 30 epochs with no divergence — no overfitting occurred, despite the small dataset, confirming the BatchNorm+Dropout combination was effective. Validation accuracy plateaued around epoch 15 (~97.6%) while train accuracy kept climbing slightly, a normal, mild generalization gap at this scale.

Notably, the model is more conservative on malignant cases (100% precision, 94% recall — it sometimes misses a malignant case) than on benign cases (96% precision, 100% recall — it never misses a benign case, but occasionally over-flags one as malignant). In a real diagnostic deployment this would be the *wrong* direction to optimize for, since false negatives on malignant cases are far more costly than false positives. This is noted here as a demonstration of understanding the clinical stakes of the metric tradeoff, even though this project is a demo, not a deployed system.

5.6 Challenges & Solutions

None encountered during implementation. The dataset is a standard, clean sklearn benchmark with no missing values or label noise, so no data-quality debugging was required. The only genuine design decision was choosing a stratified split to preserve class balance given the dataset's inherent imbalance.

---

6. CNN — Convolutional Neural Network

6.1 Dataset

FashionMNIST — 28×28 grayscale images across 10 clothing categories, loaded via `torchvision.datasets.FashionMNIST` (auto-downloads on first run). CIFAR-10 was deliberately *not* used for the base CNN task (it was reserved for Transfer Learning, Section 8) because FashionMNIST is smaller-resolution and grayscale, making it dramatically faster to train from scratch on CPU.

Subset for CPU feasibility: 8,000 of the full 60,000 training images were used (split 6,800 train / 1,200 val), plus 2,000 of the 10,000 test images. This is an explicit, documented tradeoff — full-dataset training on CPU within the project's time budget was not practical, and this is stated plainly rather than presented as a full-dataset result.

6.2 Architecture — `intermediate/cnn/model.py`

```
Input(1×28×28)
→ Conv2d(1→32, k=3) → BatchNorm2d → ReLU → MaxPool2d(2)   [28×28 → 14×14]
→ Conv2d(32→64, k=3) → BatchNorm2d → ReLU → MaxPool2d(2)  [14×14 → 7×7]
→ Flatten → Linear(64×7×7 → 128) → ReLU → Dropout(0.4) → Linear(128 → 10)
```

Two convolutional blocks, each halving spatial resolution while doubling channel depth — the standard CNN pattern of trading spatial detail for learned feature richness as depth increases. `padding=1` on both conv layers preserves spatial dimensions before pooling, so each pooling step cleanly halves the resolution.

6.3 Training Configuration

- Loss: `CrossEntropyLoss`, Optimizer: `Adam` (lr=1e-3)
- Epochs: 8, Batch size: 64
- Images normalized to mean 0.5, std 0.5 (standard grayscale normalization, not ImageNet stats since this model trains from scratch rather than using pretrained weights)

6.4 Results

Metric	Value
Test Accuracy	0.8770
Test Precision (weighted)	0.8832
Test Recall (weighted)	0.8770
Test F1 (weighted)	0.8775

Per-class F1 ranged from 0.67 (Shirt) to 0.99 (Trouser).

6.5 Observations

The most informative result here is *not* the headline accuracy but the per-class breakdown: **Shirt (67% F1) was the weakest class by a wide margin**, and Coat's precision (73%) was dragged down for the same underlying reason — Shirt, Pullover, and Coat are visually near-identical at 28×28 grayscale resolution, a well-documented FashionMNIST hard case, not a defect in this implementation. Trouser, Bag, and Ankle boot all scored above 95% F1, since these classes have distinctive silhouettes that remain distinguishable even at low resolution.

Validation accuracy fluctuated slightly rather than monotonically improving (dipped at epochs 4 and 6 before recovering) — likely a combination of the reduced training set size and the absence of learning-rate scheduling. This was not corrected, since 8 epochs was already sufficient to demonstrate a working, reasonably accurate CNN pipeline; a longer training run would benefit from an LR scheduler.

6.6 Challenges & Solutions

Compute constraint (CPU-only, no GPU): the primary constraint on this task. Solved by subsetting the training data to 8,000 images rather than the full 60,000 — sufficient to demonstrate the architecture and obtain a meaningful accuracy figure, at the cost of a few points of accuracy relative to full-dataset training. This tradeoff is documented explicitly in `results.md` rather than hidden.

---

7. RNN vs. LSTM — Sequence Models

These two models are presented together deliberately. They were trained on **identical data, identical preprocessing, and identical hyperparameters** — the only variable changed between them is the recurrent architecture itself. This was a conscious experimental design choice: rather than tuning each model independently to its own best result, holding everything else constant makes the comparison between architectures the actual point of the exercise, directly demonstrating *why* LSTM superseded vanilla RNN for practical sequence modeling.

7.1 Dataset (shared by both models)

20 Newsgroups, 4-category subset: `comp.graphics`, `rec.sport.baseball`, `sci.space`, `talk.politics.guns`, loaded via `sklearn.datasets.fetch_20newsgroups(subset='all', remove=('headers','footers','quotes'))`. Headers/footers/quoted text were stripped specifically to prevent trivial label leakage — mailing list signatures or quoted subject lines can otherwise let a model "cheat" by pattern-matching metadata rather than learning from actual content. Texts shorter than 20 characters after stripping were filtered out as noise.

Split: 70/15/15 stratified — 2,608 train / 559 val / 559 test.

7.2 Text Preprocessing — `intermediate/common/text_utils.py` (new shared module)

Built from scratch rather than depending on `torchtext` (deprecated, version-fragile) or a heavyweight tokenizer library: - **`simple_tokenize()`** — lowercases text, strips non-alphanumeric characters via regex, splits on whitespace. - **`Vocabulary`** class — builds a token-to-index mapping from the training corpus only (never val/test, to avoid leakage), capped at 10,000 tokens with a minimum frequency of 2 occurrences to exclude rare noise tokens. Reserves index 0 for `<pad>` and index 1 for `<unk>` (unknown/out-of-vocabulary tokens). - **`TextClassificationDataset`** — a PyTorch `Dataset` that encodes each text into a fixed-length (max 100 tokens) integer sequence at construction time. - **`collate_batch()`** — a custom collate function passed to `DataLoader`, using `pad_sequence` to pad variable-length sequences within a batch to the same length dynamically (rather than padding every sequence in the dataset to a fixed length up front, which wastes memory on short sequences).

This resulted in a vocabulary of 10,002 tokens (10,000 + the 2 reserved special tokens) on the actual training run.

7.3 RNN Architecture — `intermediate/rnn/model.py`

```
Embedding(10002, 100) → RNN(hidden=128, nonlinearity='tanh', 1 layer) → Dropout(0.3) → Linear(128, 4)
```

The final hidden state (`hidden[-1]`) from the RNN — not the full sequence of outputs — is passed to the classifier, following the standard many-to-one pattern for sequence classification.

7.4 LSTM Architecture — `intermediate/lstm/model.py`

```
Embedding(10002, 100) → LSTM(hidden=128, 1 layer) → Dropout(0.3) → Linear(128, 4)
```

Structurally identical to the RNN model except `nn.RNN` is replaced with `nn.LSTM`. The LSTM also returns a cell state (`cell`) alongside the hidden state; only the hidden state is used for classification, matching the RNN's usage pattern exactly for a fair comparison.

7.5 Training Configuration (identical for both)

- Loss: `CrossEntropyLoss` , Optimizer: `Adam` (lr=1e-3)
- Gradient clipping at `max_norm=5.0` — standard practice for RNN-family training stability, preventing exploding gradients during backpropagation through time
- Epochs: 10, Batch size: 32

7.6 Results — RNN

Metric	Value
Test Accuracy	**0.2719** (barely above the 25% random-guess baseline for 4 classes)
Test Precision (weighted)	0.2628
Test Recall (weighted)	0.2719
Test F1 (weighted)	0.2589

7.7 Results — LSTM

Metric	Value
Test Accuracy	**0.5492** (roughly 2× the RNN's result, same data/hyperparameters)
Test Precision (weighted)	0.5808
Test Recall (weighted)	0.5492
Test F1 (weighted)	0.5498

7.8 Observations — The Core Comparison

The RNN's training curve tells a textbook overfitting story: train accuracy climbed steadily to 59.5% by epoch 10, while validation accuracy stayed flat near the random-guess baseline (26–29%) throughout, and **validation loss increased every single epoch after epoch 1** (1.38 → 2.05). The model was memorizing training examples without learning any pattern that generalizes. This is the textbook vanishing-gradient failure mode of vanilla RNNs on longer sequences: gradients computed at the end of a 100-token sequence must propagate backward through every intermediate timestep to update early-layer weights, and for vanilla RNNs this signal degrades sharply over long sequences, making long-range dependencies effectively unlearnable.

The LSTM, trained on the exact same data with the exact same hyperparameters, nearly doubled test accuracy. Its gating mechanism (input, forget, and output gates, plus a separate cell state that carries information across timesteps with much less degradation) is specifically designed to solve this vanishing-gradient problem. The LSTM still showed some overfitting (val loss did not monotonically decrease, peaking then dipping across epochs 7–9) — but the difference in *degree* between the two models, on identical setups, is the whole demonstration.

Per-class analysis of the LSTM's results showed `rec.sport.baseball` was its strongest class (68% F1) — sports vocabulary is likely the most lexically distinct from the other three categories — while `sci.space` was weakest (33% recall despite 52% precision), plausibly due to vocabulary overlap with `comp.graphics` (both technical/scientific domains).

7.9 Challenges & Solutions

RNN's poor generalization was initially a candidate for "fixing" via hyperparameter tuning, but this was deliberately *not* pursued. The comparison between the RNN and LSTM under strictly identical conditions is the actual deliverable — tuning the RNN to a higher accuracy would have muddied the comparison and produced a less honest, less pedagogically useful result. The failure was diagnosed (vanishing gradients over long sequences), confirmed via the LSTM's improvement on the same data, and documented as the finding itself rather than papered over.

LSTM's residual overfitting (val loss instability late in training) was documented but not corrected for the same reason — correcting it would mean changing hyperparameters between the two models, breaking the controlled comparison that was the point of the exercise. A future iteration would address this via more training data, additional dropout, weight decay, or early stopping — but that is out of scope for what this comparison was designed to demonstrate.

---

8. Transfer Learning — Pretrained ResNet18 Fine-Tuning

8.1 Dataset

CIFAR-10 — 32×32 RGB images, 10 object classes, loaded via `torchvision.datasets.CIFAR10`. Subsetted to 3,400 training images (2,800 train / 600 val after an 85/15 split) and 1,000 test images, for the same CPU-feasibility reasons as the CNN task.

Images were resized from their native 32×32 to 224×224 to match ResNet18's expected input dimensions, and normalized using **ImageNet's mean/std statistics** (`[0.485, 0.456, 0.406]` / `[0.229, 0.224, 0.225]`) rather than dataset-specific statistics — this is required when using ImageNet-pretrained weights, since the pretrained convolutional filters were learned under that specific normalization and expect inputs in the same distribution.

8.2 Architecture — `intermediate/transfer_learning/model.py`

Backbone: **ResNet18**, loaded with `ResNet18_Weights.IMAGENET1K_V1` pretrained weights. The transfer learning strategy applied: 1. **Freeze all layers** initially (`requires_grad = False` on every parameter). 2. **Unfreeze only** `layer4` (the final residual block) — allowing the highest-level, most task-specific features to adapt to CIFAR-10's classes, while leaving the early layers (which encode generic, broadly-transferable features like edges and textures) untouched. 3. **Replace the final classifier head** (`fc` layer) with a new `Linear(512, 10)` layer for CIFAR-10's 10 classes — this layer is always trainable regardless of the freezing strategy, since it did not exist in the original ImageNet model (which had 1,000 output classes).

This is the standard, well-established transfer learning pattern: reuse generic low-level features as-is, retrain only the task-specific upper layers. The `get_trainable_params()` method on the model class reports the exact trainable/total parameter split for transparency in results reporting.

Actual measured split on this run: 8,398,858 trainable / 11,181,642 total parameters (75.1% trainable).

8.3 Training Configuration

- Loss: `CrossEntropyLoss`
- Optimizer: `Adam`, learning rate **1e-4** (deliberately lower than the 1e-3 used elsewhere — fine-tuning pretrained weights requires smaller update steps than training from scratch, to avoid destructively overwriting the useful pretrained features)
- Epochs: 6, Batch size: 32

8.4 Results

Metric	Value
Test Accuracy	0.8410
Test Precision (weighted)	0.8425
Test Recall (weighted)	0.8410
Test F1 (weighted)	0.8409

Per-class F1 ranged from 0.71 (cat, dog) to 0.94 (automobile).

8.5 Observations

Convergence was dramatically faster than the from-scratch CNN in Section 6: train accuracy hit 93% by epoch 2 and 99%+ by epoch 3. This is the expected, direct benefit of transfer learning — the pretrained ImageNet features already encode strong general visual representations (edges, textures, shapes, basic object parts), so the model only needs to learn the mapping from those existing features to CIFAR-10's specific 10 classes, rather than learning visual feature extraction from scratch.

This fast convergence came with a clear cost: **overfitting from epoch 3 onward**. Train loss collapsed toward zero (0.10 → 0.02) while validation loss plateaued and even slightly worsened (0.4767 → 0.4830), and validation accuracy stopped improving despite train accuracy sitting near-perfect. With 75.1% of the model's parameters trainable and only 2,800 training images, the model had far more capacity than the dataset size could responsibly support — it had enough room to memorize the training set well before it stopped improving on unseen data.

Cat (71% F1) and dog (71% F1) were the weakest classes — a well-known CIFAR-10 hard pair, since cats and dogs share substantial visual overlap (fur texture, four-legged pose, similar indoor/outdoor backgrounds) compared to classes with more distinct shapes, like automobile, truck, or ship.

Despite the overfitting, 84.1% test accuracy from only 2,800 training images is a strong result and is the core value proposition of transfer learning made concrete: high accuracy achievable with a fraction of the data a from-scratch CNN would require to reach comparable performance.

8.6 Challenges & Solutions

Overfitting from high trainable-parameter count relative to small dataset size: identified via the train/val loss divergence starting at epoch 3. This was documented honestly rather than re-tuned to hide it. A future iteration would freeze `layer4` as well (leaving only the small classifier head trainable, a much smaller parameter count) or add dropout before the final linear layer — trading some accuracy ceiling for better generalization.

DNS resolution failure downloading ResNet18's pretrained weights on the first run attempt (`download.pytorch.org` failed to resolve, while CIFAR-10's download from a different host succeeded in the same session) — a transient network issue, not a code defect. Resolved by simply retrying the script; CIFAR-10 was already cached locally from the first attempt, so only the ~45MB weights file needed to re-download. Noted here as a known flaky point for anyone reproducing this project. ## 9. Advanced Level — RAG Chatbot

9.1 System Architecture

The full pipeline, end to end:

```
[User uploads a document: PDF / DOCX / TXT / XLSX]
```

```
        ↓
[Ingestion Layer — advanced/ingestion/loaders.py]
   format-specific loader dispatches based on file extension
        ↓
[Chunking — advanced/ingestion/chunking.py]
   RecursiveCharacterTextSplitter, 1000 chars/chunk, 200 char overlap
        ↓
[Embedding — advanced/embeddings/embedder.py]
   sentence-transformers (all-MiniLM-L6-v2), local, CPU
        ↓
[ChromaDB — advanced/retrieval/retriever.py]
   persisted vector store, chunks stored with source metadata
        ↓
[User asks a question]
        ↓
[Query Rewriting — advanced/chain/rag_chain.py]
   follow-up questions rewritten into standalone questions using chat history
        ↓
[Retrieval — top-k similarity search against ChromaDB]
        ↓
[Relevance Threshold Check]
   if no chunk scores above SIMILARITY_THRESHOLD (0.3) → return fallback, skip LLM call entirely
        ↓
[Prompt Construction — retrieved chunks + question, NOT chat history (see 9.3)]
        ↓
[Groq LLM — llama-3.3-70b-versatile]
   temperature=0.2 (low — factual grounding prioritized over creative variation)
        ↓
[Response + Source Attribution]
   answer text + list of source filenames the answer was grounded in
        ↓
[Memory Update — advanced/memory/conversation_memory.py]
   (question, answer) pair appended, oldest turn dropped if over MAX_CHAT_HISTORY_TURNS
```

9.2 File-by-File Breakdown

`advanced/config.py`

Centralizes every tunable value in the system so nothing is a magic number buried in application logic. Loads the Groq API key from a `.env` file via `python-dotenv`, and **hard-fails at import time with a clear error message if the key is missing** — this was a deliberate choice: failing loudly and immediately at startup is better than failing confusingly later, deep inside a Groq API call, with a cryptic authentication error.

Also defines: ChromaDB persistence directory, chunk size (1000 characters) and overlap (200 characters), the embedding model name, retrieval `k` (4 — how many chunks are retrieved per query), the similarity threshold (0.3) below which retrieved chunks are discarded as irrelevant, and the maximum number of conversation turns retained in memory (5).

`advanced/ingestion/loaders.py`

The multi-format ingestion layer. Defines two custom exception classes — `UnsupportedFileTypeError` and `DocumentLoadError` — so calling code can distinguish "this file type isn't supported" from "this file is corrupt/empty/unreadable," which matters for surfacing the correct user-facing error message in the UI later.

Four format-specific loader functions: - **`load_pdf()`** — wraps LangChain's `PyPDFLoader`. Explicitly checks whether the loaded document has any extractable text at all, raising `DocumentLoadError` if not — this catches the case of a scanned/image-only PDF, which would otherwise silently produce an empty document that fails mysteriously later in the pipeline rather than with a clear error immediately. - **`load_docx()`** — wraps LangChain's `Docx2txtLoader` (which itself depends on the separate `docx2txt` package — see Section 9.4 for the dependency issue this caused). - **`load_txt()`** — wraps LangChain's `TextLoader`, explicitly catching `UnicodeDecodeError` to give a clear message if a file isn't valid UTF-8, rather than letting a cryptic encoding traceback surface. - **`load_xlsx()`** — hand-rolled directly with `openpyxl`, not a LangChain built-in loader. This was a deliberate choice: no off-the-shelf LangChain Excel loader reliably handles arbitrary sheet layouts (multiple sheets, mixed data types, empty rows) without configuration. The hand-rolled version iterates every sheet, converts each row to a pipe-delimited text line, and skips fully-empty rows, producing one `Document` object per non-empty sheet with the sheet name attached as metadata.

A `LOADER_MAP` dictionary dispatches from file extension to the correct loader function, and the main entry point `load_document()` validates the file exists, checks the extension is supported, calls the appropriate loader, and stamps every returned document with a consistent `source_file` metadata field (the filename) — used later for citation display.

`advanced/ingestion/chunking.py`

Uses LangChain's `RecursiveCharacterTextSplitter`, configured with a separator priority list: `["\n\n", "\n", ". ", " ", ""]` — the splitter tries to break on paragraph boundaries first, falling back to line breaks, then sentence boundaries, then word boundaries, and only as a last resort splits mid-word. This preserves semantic coherence within chunks far better than a naive fixed-character-count split, which could otherwise cut a sentence in half mid-thought.

Chunk size of 1000 characters with 200 characters of overlap between consecutive chunks — the overlap ensures that information sitting near a chunk boundary isn't lost from context entirely; it appears at the tail of one chunk and the head of the next. Each chunk is also stamped with a `chunk_index` (its position within its source document) for citation display purposes.

`advanced/embeddings/embedder.py`

A thin wrapper around LangChain's `HuggingFaceEmbeddings`, loading `all-MiniLM-L6-v2` on CPU with `normalize_embeddings=True`. Normalizing embeddings to unit length is what makes cosine similarity (the standard metric for comparing text embeddings) behave cleanly and consistently. Kept as a separate, single-purpose module specifically so the rest of the application depends on this interface rather than directly on LangChain's embedding class — if the embedding model needs to change later, this is the only file that needs to change.

`advanced/retrieval/retriever.py`

Manages the ChromaDB vector store lifecycle: - **`get_vector_store()`** — returns a `Chroma` instance bound to a named collection, persisted to disk (not in-memory), so indexed documents survive between application restarts. - **`index_documents()`** — embeds and stores a list of chunked documents into the vector store, raising `ValueError` if given an empty chunk list rather than silently doing nothing. - **`retrieve_relevant_chunks()`** — runs `similarity_search_with_relevance_scores()` against the store, returning `(Document, score)` tuples. Wrapped in a try/except that returns an empty list on failure (e.g., an uninitialized/empty collection) rather than crashing — an empty result is then correctly interpreted downstream as "nothing relevant found," which is the correct behavior, not a special case requiring separate handling. - **`clear_collection()`** — deletes all documents in a named collection, used when starting a fresh session or during testing.

`advanced/memory/conversation_memory.py`

A deliberately simple, hand-rolled memory implementation — a plain Python list of `{"question": ..., "answer": ...}` dictionaries — rather than using LangChain's built-in memory classes. This was a considered decision: LangChain's memory API has been in significant flux across recent versions (multiple classes deprecated or restructured), and a plain list is more predictable, easier to debug, and has zero risk of breaking due to an unrelated LangChain version bump. `add_turn()` appends a new exchange and drops the oldest turn if the history exceeds `MAX_CHAT_HISTORY_TURNS` (5); `get_formatted_history()` renders the history as plain text for injection into prompts.

`advanced/chain/rag_chain.py`

The orchestration core, tying every other module together. Detailed in Section 9.3 below given the significant debugging work involved.

9.3 The RAG Chain — Design and a Real Bug Fixed

The `RAGChain` class exposes one primary method, `ask(question)`, which:

1. **Rewrites the query if conversation history exists.** This is the single most important design decision in the chain, added *after* discovering it was missing during testing (documented honestly below, not smoothed over).
2. **Retrieves relevant chunks** using the rewritten (not the original) question, via `retrieve_relevant_chunks()`.
3. **Filters retrieved chunks by the relevance threshold** (0.3). If nothing survives the filter, the chain **returns a fixed fallback message and never calls the LLM at all** — this is the core anti-hallucination guarantee. A weaker implementation might call the LLM anyway with an empty or irrelevant context and hope the model says "I don't know" on its own; this implementation makes that outcome structurally impossible by never giving the

LLM the opportunity to guess.

4. **Builds the prompt** from the retrieved chunks (each labeled with its source filename) and the *original* user question — deliberately using the original question here, not the rewritten one, so the LLM responds to what the user actually typed rather than a machine-rewritten paraphrase.

5. **Calls Groq** with a system prompt that explicitly instructs the model to answer only from the provided context, to say so explicitly if the context is insufficient, and to cite sources — at `temperature=0.2`, a low value chosen because factual grounding matters more than response variety for this task.

6. **Updates memory** with the (question, answer) pair, then returns a dict containing the answer, the list of source filenames, and a `grounded` boolean flag indicating whether the fallback was used.

The bug: follow-up questions failed silently

During smoke testing (three sequential questions against a single indexed document about the Eiffel Tower), the first version of the chain produced this behavior:

- **Q1:** *"When was the Eiffel Tower completed?"* → correctly answered "1889," grounded, source cited.
- **Q2:** *"How tall is it?"* → **incorrectly** returned the "no relevant information found" fallback.
- **Q3:** *"What is the capital of Japan?"* (deliberately unrelated) → correctly returned the fallback.

Q3 working correctly while Q2 failed was the diagnostic clue. Q2 is a natural follow-up question — "it" only resolves to "the Eiffel Tower" given the context of Q1. But the *retrieval step ran on the raw text "How tall is it?"* before any conversation history was involved. Embedded and searched against ChromaDB in isolation, that question has weak semantic similarity to a document about the Eiffel Tower (no shared vocabulary at all beyond "tall"), so it scored below the 0.3 relevance threshold and was rejected — the system never even reached the point where conversation history could have helped, because the fallback fired before generation was ever attempted.

This is a well-known failure mode in naive RAG implementations: **retrieval and generation are two separate steps, and only generation was history-aware in the initial design — retrieval was not.**

The fix: added a query-rewriting step (`_rewrite_query()`) that runs *before* retrieval whenever conversation history exists. It sends the conversation history plus the new question to Groq with an explicit instruction to rewrite ambiguous follow-ups into standalone questions (e.g., "How tall is it?" → "How tall is the Eiffel Tower?"), and passes that rewritten query into the retrieval step instead of the raw question. If it's the first question in a session (no history yet), this step is skipped entirely, adding zero extra API calls or latency on the common case.

After the fix, the same three-question test produced: Q1 correct and grounded, **Q2 correctly rewritten and correctly grounded ("The Eiffel Tower is 330 meters tall")**, Q3 still correctly rejected via the fallback. This confirms the fix solved the actual problem without breaking the fallback behavior it needed to coexist with.

This bug and fix are included here in detail because it's a genuine, non-trivial RAG architecture lesson — not a typo or a version mismatch — and understanding *why* it happened (retrieval and generation need to share the same context-awareness, not just generation) is more valuable to demonstrate than pretending the first implementation was correct from the start.

9.4 Streamlit UI

The interface (`advanced/app/streamlit_app.py`) is a single-file Streamlit application providing document upload, a chat interface, and source citation display. Key design decisions:

Session-scoped ChromaDB collections. Each browser session is assigned a unique UUID at first load, and all indexed documents go into a collection named `session_{uuid}` rather than a single shared collection. This means opening the app in two separate browser tabs (or two people using a shared deployment) never causes documents or conversation context to bleed between sessions — a real correctness concern for a multi-user-capable interface, even though this project was tested single-user.

Upload flow and error surfacing. Uploaded files arrive as in-memory buffers from Streamlit's file uploader widget, but the ingestion layer's loaders (Section 9.2) expect filesystem paths. Each upload is written to a temporary file via `tempfile.NamedTemporaryFile`, processed through the existing `load_document()` → `chunk_documents()` → `index_documents()` pipeline unchanged, and the temp file is deleted in a `finally` block regardless of whether processing succeeded — so failed uploads never leak temp files. Errors from the ingestion layer (`UnsupportedFileTypeError`, `DocumentLoadError`) are caught explicitly and surfaced via Streamlit's `st.error()` with the

original, specific error message, rather than a generic “upload failed.”

Chat state and re-indexing guard. The app tracks which filenames have already been indexed in `st.session_state.indexed_files`, so re-running the upload widget (which Streamlit does on every interaction) doesn’t re-embed and re-index the same file repeatedly.

Interaction with the RAG chain. The chat input is disabled (`st.chat_input(..., disabled=not st.session_state.indexed_files)`) until at least one document has been successfully indexed, preventing a confusing “why won’t it answer” state. Each question is passed to the existing, unmodified `RAGChain.ask()` method — the UI layer adds no RAG logic of its own; it is purely a presentation layer over the chain built and tested in Section 9.3.

9.5 End-to-End Verification

With the UI complete, the full pipeline was verified in the browser (not just via terminal smoke tests) using the project’s own technical report PDF as the test document — a real, substantial, multi-section document rather than a toy fixture. Three questions were asked in sequence:

1. **Grounded question**: *“What was the RNN’s test accuracy?”* → Correctly answered 0.2719, with the source document and section number cited.
2. **Follow-up question** (testing query rewriting live in the UI, not just in isolation): *“How does that compare to the LSTM?”* → Correctly resolved “that” to the RNN’s accuracy, retrieved the LSTM’s result (0.5492), and stated the roughly 2x relationship — confirming the query-rewriting fix from Section 9.3 functions correctly end-to-end through the full UI, not only in a controlled backend test.
3. **Unrelated question**: *“What’s the capital of France?”* → Correctly returned the fallback message, refusing to answer from general knowledge despite the LLM certainly “knowing” the answer — confirming the anti-hallucination guarantee holds through the complete user-facing flow.

All three behaviors matched the backend smoke test results from Section 9.3 exactly, confirming the UI layer introduced no regressions in retrieval, generation, memory, or the fallback logic. ## 10. Consolidated Challenges and Solutions Log

This section pulls every challenge encountered across the entire project into one place, in chronological order, since several of them (particularly the Git incidents) are infrastructure problems that cut across both the Intermediate and Advanced levels rather than belonging to any single model.

10.1 Library Version Drift (LangChain ecosystem)

Problem: `from langchain.text_splitter import RecursiveCharacterTextSplitter` failed with `ModuleNotFoundError` — the installed LangChain version had moved text splitters into a separate package, `langchain_text_splitters`, deprecating the old import path. **Solution:** Updated the import to `from langchain_text_splitters import RecursiveCharacterTextSplitter`, installed the package directly, and pinned it explicitly in `requirements.txt` to prevent a clean install from hitting the same gap. **Lesson:** LangChain’s package structure has been actively splitting into smaller, more focused packages across recent releases. Anyone reproducing this project should expect similar import-path issues if using a different LangChain version than the one pinned in `requirements.txt`.

10.2 Missing Transitive Dependency (docx2txt)

Problem: `Docx2txtLoader.load()` failed with `ModuleNotFoundError: No module named 'docx2txt'`. LangChain’s DOCX loader depends on the separate `docx2txt` package, which is distinct from `python-docx` (already installed, used elsewhere for writing test fixtures) — an easy dependency to miss since the two package names are easily confused. **Solution:** Installed `docx2txt` directly and added it to `requirements.txt`. **Note on error handling:** this failure was caught cleanly by the `DocumentLoadError` exception wrapper already built into `loaders.py` — the test suite reported a clear, informative failure message rather than a raw unhandled traceback, confirming the error-handling design was working as intended even while surfacing a real environment issue.

10.3 Two Git Large-File Incidents

Incident 1: After committing the Transfer Learning implementation, `git push` was rejected by GitHub — `cifar-10-python.tar.gz` (162.60 MB) exceeded GitHub’s 100 MB file size limit. Root cause: the original `.gitignore` only excluded `data/sample_datasets/*.zip`, not `.tar.gz` archives or the raw extracted dataset folders (`FashionMNIST/`, `cifar-10-batches-py/`) that `torchvision`’s dataset downloaders create automatically. Both FashionMNIST (from the CNN task) and CIFAR-10 (from the Transfer Learning task) had already been committed into git history across two separate

commits by the time this was caught.

Fixing this required more than simply deleting the files and re-committing, since they were already baked into prior commits' history — deleting them in a new commit would leave the large blobs sitting in the repository's history regardless, and `git push` would still fail. The fix used `git filter-repo` (installed via `pip install git-filter-repo` inside the project's virtual environment) to rewrite the entire commit history, stripping the offending paths out of every commit rather than just the most recent one:

```
git filter-repo --path data/sample_datasets/FashionMNIST \
                --path data/sample_datasets/cifar-10-batches-py \
                --path data/sample_datasets/cifar-10-python.tar.gz \
                --invert-paths --force
```

`.gitignore` was corrected in the same pass to exclude `data/sample_datasets/**/*` (with a `.gitkeep` exception to preserve the empty folder structure) and `*.tar.gz`, preventing recurrence. Since `filter-repo` removes the `origin` remote as a safety measure, it had to be re-added afterward, and the rewritten history required a force-push (safe in this case specifically because the original push had never succeeded — no collaborator or CI system had any dependency on the old, contaminated history).

Incident 2 (a genuine process failure, documented honestly): Despite the fix above, the *same* CIFAR-10 archive reappeared in a subsequent commit and blocked a later push. Root cause: the corrected `.gitignore` content had been written and shown, but was never actually saved to disk and committed — the file on disk still contained the original, incomplete version. This was only caught because the push failed a second time and the commit history was re-inspected (`git log --oneline -- <path>`) to confirm which specific commit reintroduced the file.

Process change adopted going forward: any `.gitignore` modification is now confirmed as saved and its full content is verified *before* any subsequent `git add` or `git commit` command is issued — closing the gap that allowed this to happen twice. This is documented here deliberately, including the repeat occurrence, because it reflects a real process improvement made mid-project rather than a one-off fluke.

10.4 RAG Follow-Up Question Failure (Query Rewriting)

Covered in full in Section 9.3 — the most substantial debugging work in the RAG portion of the project. Summarized here for completeness: follow-up questions using pronouns ("how tall is it?") failed to retrieve relevant context because retrieval ran on the raw question text, without access to conversation history that would have resolved the pronoun. Fixed by adding a query-rewriting step using Groq to convert history-dependent follow-ups into standalone questions before retrieval, while still using the original question (not the rewritten version) for final answer generation.

10.5 Compute Constraints (CPU-only hardware, throughout)

No GPU was available for any part of this project. This shaped concrete decisions across every model that touches image or larger-scale data: FashionMNIST and CIFAR-10 were both subsetted rather than trained on their full sizes; Transfer Learning used a lower learning rate and fewer epochs appropriate to fine-tuning rather than from-scratch training; and dataset/epoch choices throughout favored CPU-feasible runtimes over maximizing raw accuracy. Every instance of this tradeoff is called out explicitly in the relevant model's results section rather than presented as an unconstrained result.

10.6 Transient Network Failure (ResNet18 weights download)

A DNS resolution failure (`getaddrinfo failed`) occurred when downloading ResNet18's pretrained ImageNet weights from `download.pytorch.org`, while CIFAR-10's download from a different host succeeded in the same session moments earlier — indicating a transient, host-specific network blip rather than a systemic connectivity problem. Resolved by simply retrying the script.

---

11. Installation and Setup Guide

11.1 Prerequisites

- Python 3.11+ (project developed and tested on Python 3.11.6)
- Git

- A Groq API key (free tier available at console.groq.com)
- Internet access for initial dataset downloads (FashionMNIST ~30MB, CIFAR-10 ~170MB, ResNet18 weights ~45MB, sentence-transformers model ~90MB) — all cached locally after first run

11.2 Clone and Environment Setup

```
git clone https://github.com/SameerAhmedAI/DL-RAG-toolkit.git
cd DL-RAG-toolkit

python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS/Linux
source .venv/bin/activate

pip install -r requirements.txt
```

11.3 Environment Variables

Create a `.env` file at the repository root:

```
GROQ_API_KEY=your_groq_api_key_here
```

The RAG chatbot's `config.py` will raise a clear error at startup if this is missing — this is intentional fail-fast behavior, not a bug.

11.4 Running the Intermediate Level Models

Each model is run as a module from the repository root (not from inside its own folder):

```
python -m intermediate.ann.train
python -m intermediate.cnn.train
python -m intermediate.rnn.train
python -m intermediate.lstm.train
python -m intermediate.transfer_learning.train
```

Each script trains the model, saves a checkpoint and result plots to that model's `results_artifacts/` folder, and prints a full evaluation summary to the terminal. CNN and Transfer Learning will auto-download their datasets on first run.

11.5 Running the RAG Chatbot Tests

```
pytest advanced/tests/test_ingestion.py -v
```

11.6 Running the RAG Chatbot UI

```
streamlit run advanced/app/streamlit_app.py
```

Opens automatically in the default browser. Upload a PDF, DOCX, TXT, or XLSX file via the sidebar, wait for the "Indexed N chunks" confirmation, then ask questions in the chat panel. Answers are grounded strictly in the uploaded document(s); questions outside their scope return an explicit fallback message rather than a general-knowledge answer.

11.7 Known Setup Issues

- If `git filter-repo` is needed for any reason, it must be installed via `pip install git-filter-repo` — it is not a built-in git command.
- If a `ModuleNotFoundError` occurs for any LangChain-related import, check whether the specific submodule has been split into its own package in the installed LangChain version (see Section 10.1) — this project pins working

versions in `requirements.txt`, but LangChain's package structure continues to evolve.

---

12. Future Improvements

Intermediate Level: - Train CNN and Transfer Learning models on their full datasets (given GPU access) to establish an accuracy ceiling above the CPU-constrained subset results reported here. - Add learning rate scheduling to the CNN and LSTM training loops to address the validation instability observed in both. - Extend the RNN/LSTM comparison with a GRU variant, adding a third point of comparison on the same controlled dataset. - Reduce the Transfer Learning model's trainable parameter count (freeze `layer4` as well, leaving only the classifier head trainable) to address the overfitting observed in Section 8.5, trading peak accuracy for better generalization.

Advanced Level (RAG Chatbot): - Add support for multi-document cross-referencing in a single answer, with per-source attribution for each claim rather than a single combined source list. - Add evaluation metrics for the RAG pipeline itself (e.g., retrieval precision/recall against a labeled question set), not just qualitative smoke testing. - Consider a re-ranking step after initial retrieval (e.g., a cross-encoder) to improve precision on ambiguous queries, particularly relevant given the relevance-score calibration warning noted during retrieval testing (Chroma's relevance scores occasionally fell outside the expected 0–1 range on small demo datasets). - Add persistent, named chat sessions (currently conversation memory exists only within a single `RAGChain` instance's lifetime).

---

13. References

- PyTorch documentation — pytorch.org/docs
- torchvision datasets and pretrained models — pytorch.org/vision
- scikit-learn documentation — scikit-learn.org
- LangChain documentation — python.langchain.com
- ChromaDB documentation — docs.trychroma.com
- Groq API documentation — console.groq.com/docs
- sentence-transformers documentation — sbert.net
- FashionMNIST dataset — Xiao, Han, et al. "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms." (2017)
- CIFAR-10 dataset — Krizhevsky, Alex. "Learning Multiple Layers of Features from Tiny Images." (2009)
- 20 Newsgroups dataset — scikit-learn built-in loader, originally compiled by Ken Lang
- Breast Cancer Wisconsin (Diagnostic) dataset — UCI Machine Learning Repository / scikit-learn built-in loader
- ResNet architecture — He, Kaiming, et al. "Deep Residual Learning for Image Recognition." (2015)